COMP 3920 Database Systems 1

MySQL
Summary Tables

Introduction:
The goal of this document is to serve as a walkthrough working with Summary Tables, in MySQL 8.

We will use the database from a previous week called **lotsofdata**.

What happens if indexes aren't enough? In some cases, there is just too much data and the queries that access that data need to access too much of it at once that regardless of the index(es) these queries will be too slow. This typically happens when running extensive reports that summarize large amounts of data. What can we do then?

Depending on your data, the queries you need to run on the data, and your specific use case, it may be possible to create summary tables. Summary tables, take large amounts of data, and perform calculations such as counts, min, max, average, etc. (aggregation functions) and cache them for future use. Using this cache can speed up your queries immensely.

GOAL of Summary Tables:
Our goal of summary tables is to reduce the number of rows we need to look at to get the data we need, while still providing enough detail that our queries are worthwhile.

We must carefully choose columns to do a group by to populate our summary tables. Let's say our original table has 10,000,000 (10 million) rows. If our group by column(s) is/are too specific and our summary table has 2,000,000 (2 million) rows, we haven't summarized our table enough and it will take almost as long to query our summary table as it did the original table. If our group by column(s) are not specific enough, we will lose lots of information about the original table. The most extreme example is performing a SELECT count(*) FROM <table>; This is a summary table, and does provide some information, but could be more beneficial if it where grouped by created_date, so that we could see the count on a daily basis. Once we have the data on a daily basis it is easy to see how we could get the information on a weekly, monthly, quarterly or yearly basis.

Example:

Data - Employee_Sales

Sale_Time	Employee	Product	Amount
2018-11-05 11:05 AM	Frank	Remote Control Train Set	$1,058.12
2018-11-05 1:05 PM	Frank	Remote Control Car	$298.23
2018-11-05 2:05 PM	Frank	Jigsaw Puzzle	$50.76
2018-11-05 3:43 PM	Frank	Rubics Cube	$20.48
2018-11-05 5:12 PM	Frank	Fullsize Batman Replica	$764.52
2018-11-05 1:45 PM	Sara	Coloring Books	$72.71
2018-11-05 12:06 PM	Sara	Kids Racing Card Bed	$1,208.55
2018-11-06 2:44 PM	James	Board Games	$62.64
2018-11-06 4:21 PM	James	Lego Sets	$232.66
2018-11-06 4:57 PM	James	Animal Costumes	$158.22
2018-11-06 3:52 PM	Sara	Glitter Glue Craft Set	$44.22
2018-11-06 5:32 PM	Sara	Lego Sets	$106.77

Summary Data –
Daily_Employee_Sales_Totals

Sales_Day	Employee	Total	SalesCount
2018-11-05	Frank	$2,192.11	5
2018-11-05	Sara	$1,281.26	2
2018-11-06	James	$453.52	3
2018-11-06	Sara	$150.99	2

In this example, we have reduced the amount of data in the original table, while still providing the ability to produce even further summary data, such as:

- Daily Sales (all Employees, or on specific employees)
- Weekly Sales
- Monthly Sales
- Yearly Sales

The summary table is grouped by Day(SaleTime) and Employee. The Total column is our SUM() aggregate which totals the sales for that employee on that day.

The next challenge is how do we maintain the summary tables with accurate information?

There are a number of techniques that can be used:

1. Triggers
2. External Code
3. Scheduled Event
4. On Demand

1. Triggers. We have seen in our Triggers Lab, that we can maintain an employee row count by using triggers on the INSERT and DELETE events. This concept could be expanded and applied to summary tables so that any change to the base table would update the summary table keeping it accurate.
2. External Code. Similar, to Triggers, we can write code in what ever language our platform is using (for example: PHP, Python, C#, Java, etc.) that whenever it updates our base table, it also updates our summary tables.
3. Scheduled Event. In the Scheduled Event Lab, we have seen how we can create tasks that run periodically. We can create a daily, weekly, monthly or yearly task that will summarize the data for the previous period. For example, on the first of November we could summarize the data for all of October and store it into the summary tables. This event would get scheduled on the first of every month.
4. On Demand. The summary table could be generated the first time someone requests the data for that period. Once created, it would be cached so others who use the data later wouldn't need to re-generate it.

In general, using a created_date is very useful so that we can keep the previously calculated information.

When inserting into summary tables it is often useful to use the `INSERT … ON DUPLICATE KEY UPDATE` statement.

In the previous example with our Summary Table – Daily Employee Sales Totals, we can create a unique key on Sales_Day and Employee – this way we know that there will not be duplicate entries for the same employee on the same day. Each time we need to insert new data into the Summary Table we assume it's a new row and INSERT as usual. If that Employee and Day already exist, we use this ON DUPLICATE KEY UPDATE statement to increment our Total column.

Example:

```
INSERT INTO
Daily_Employee_Sales_Totals
SELECT DATE(Sale_Time), Employee, SUM(Amount)
FROM Employee_Sales
GROUP BY DATE(Sale_Time), Employee
ON DUPLICATE KEY UPDATE Total = Total + Amount;
```

This IODKU (Insert On Duplicate Key Update) could be rewritten as (in pseudocode):

Query_1:
```
SELECT Sales_Day, Employee
FROM Daily_Employee_Sales_Totals
WHERE SALES_DAY = "2018-11-10"
AND Employee = 'Frank'

if (Query_1.rowCount == 1) {
      Query_2:
      SELECT DATE(Sale_Time) AS Sales_Day,
            Employee,
            SUM(Amount) AS DailyTotal
      FROM Employee_Sales
      GROUP BY DATE(Sale_Time), Employee;

      Query_3:
      UPDATE
      Daily_Employee_Sales_Totals
      SET TOTAL = TOTAL + Query_2.DailyTotal
      WHERE SALES_DAY = Query_2.Sales_Day
      AND Employee = Query_2.Employee
}
else {
      Query_4:
      INSERT INTO
      Daily_Employee_Sales_Totals
      SELECT DATE(Sale_Time), Employee, SUM(Amount)
      FROM Employee_Sales
      GROUP BY DATE(Sale_Time), Employee;
}
```

The IODKU is much more concise.

Step 1:
Login to MySQL as user `root`.
Switch the default database to `lotsofdata`.

Step 2:
Create a summary table for the `user` table called `user_summary`:

```
CREATE TABLE `user_summary` (
  `created_year` int DEFAULT NULL,
  `lastname_initial` varchar(1) DEFAULT NULL,
  `birth_year` int DEFAULT NULL,
  `rowcount` bigint NOT NULL DEFAULT '0',
  UNIQUE KEY `unique_summary`
(`created_year`,`lastname_initial`,`birth_year`)
) ENGINE=InnoDB;
```

Note the unique key `unique_summary` that there will only ever be a single record for `created_year`, `lastname_initial` and `birth_year`.

Step 3:
Now let's insert our data into our summary table:

```
INSERT INTO user_summary
SELECT
 year(created_date),
 left(lastname,1),
 year(birth_date),
 count(*)
FROM user
GROUP BY year(created_date),
 left(lastname,1),
 year(birth_date);
```

Count the rows in the base table and take note of the execution time:
Base Table Count Query:

```
SELECT count(*) from user;
```

Now, let's derive the number of rows from the base table by using the summary table:
We can sum up the individual counts for each of the rows and get back the original row count:
Summary Table Count Query:

```
SELECT sum(rowcount) from user_summary;
```

Notice the number of rows from the Base Table Count Query and the Summary Table Count Query should match, but the execution time of the Summary Table Count Query should be much faster.

Submit your screenshot (including execution time):

04_Base_Table_Count_Query.jpg

Submit

```
mysql> select count(*) from user;
+----------+
| count(*) |
+----------+
|          |
+----------+
1 row in set (      sec)
```

Submit your screenshot (including execution time):

05_Summary_Table_Count_Query.jpg

Submit

```
mysql> select sum(rowcount) from user_summary;
+---------------+
| sum(rowcount) |
+---------------+
|               |
+---------------+
1 row in set (      sec)
```

Step 4:

Let's see how well we did at reducing the number of rows from our base table to our summary table:

```
SELECT count(*) from user_summary;
```

```
mysql> select count(*) from user_summary;
+----------+
| count(*) |
+----------+
|    29946 |
+----------+
1 row in set (0.02 sec)
```

This is good! We've gone from 8,000,000 rows down to 29,946. This is a reduction factor of 276x (8,000,000 ÷ 29,946 = 276.38). Take note that a reduction factor that is too low (ex: 2x to 10x) may do little, if anything, to reduce our execution times. On the flip-side, if our reduction factor is too high (100,000x or more) we may lose so much of the detail we need to retain in order to provide any useful information, i.e. we could do a reduction factor of 8,000,000x (by calculating the row count) but it wouldn't provide any more information other than that!

Step 5:
Create a few summary queries that calculate the following:

1. How old is the oldest user (show their age)?
2. Most popular age for all users.
3. Find the least popular years (bottom 5) for new user signups (excluding 2000 and 2018).
4. Calculate the most popular (top 5) lastname initials of all time.

HINT: Writing queries for summary tables is sometimes different than running the same query on the base table. Should you be using count(*) or sum(rowcount) on the summary table queries?

Submit a screenshot of each of the above queries (1-4) that includes the query, results and execution time.

1. **06_oldest_user.jpg** Submit
2. **07_popular_user_age.jpg** Submit
3. **08_least_popular_year.jpg** Submit
4. **09_popular_lastname_initials.jpg** Submit

Example (shows query, results and execution time):

```
mysql> select created_year, lastname_initial, sum(rowcount)from user_summary grou
p by created_year, lastname_initial limit 10;
+--------------+------------------+---------------+
| created_year | lastname_initial | sum(rowcount) |
+--------------+------------------+---------------+
|         2000 | A                |          6019 |
|         2000 | B                |         21573 |
|         2000 | C                |         19043 |
|         2000 | D                |          9122 |
|         2000 | E                |          3365 |
|         2000 | F                |          9152 |
|         2000 | G                |         11037 |
|         2000 | H                |         19160 |
|         2000 | I                |           353 |
|         2000 | J                |          4649 |
+--------------+------------------+---------------+
10 rows in set (0.00 sec)
```